SIMATS ENGINEERING

OBJECT ORIENTED PROGRAMMING USING C++

ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading

Student Name	NELLEPALLI VINAY YADAV
Register Number	192472061
Department / Section	CSE-AI
Submission Date	13/11/2026
GitHub Repository Link	https://github.com/VINAYYADAV10/DSA0117---OBJECT-ORIENTED-PROGRAMMING-WITH-C-
Compiler used	g++ (Ubuntu 13.3.0) 13.3.0
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file

Note: This report provides complete reference implementations and reproducible output examples. Run every source file on your own system before submission. Timing values depend on the computer and compiler; record the values printed by your own runs rather than copying invented timings.

Repository, Compiler and Execution Details

Create one folder for each task (Task01 to Task07). Save the corresponding source code in that folder, compile it with the command shown below, run it, and save a screenshot of the terminal output. Add at least one progressive Git commit for each task.

Suggested repository structure

```
DSA01-192471055-Assignment3/
  README.md
  Task01/ constructors.cpp
  Task02/ prescription.cpp  shallow_copy_demo.cpp
  Task03/ auditorium.cpp
  Task04/ money.cpp
  Task05/ time_playlist.cpp
  Task06/ text_class.cpp
  Task07/ complex_impedance.cpp
```

Compile, run and Git commands

```
g++ --version
g++ -std=c++17 -Wall -Wextra constructors.cpp -o constructors
./constructors
git init
git add Task01
git commit -m "Task01: constructors and constructor overloading"
# Repeat add/commit for Task02 through Task07
git remote add origin https://github.com/<username>/DSA01-192471055-Assignment3.git
git push -u origin main
```

README.md should contain your name, register number, compiler/version, compile command, run commands, a short result for each task, and the working GitHub repository URL.

Q1 – Constructors and Constructor Overloading (6 Marks)

1. Problem Analysis and Design

A library needs Book records for three cases: an empty record, a donated book with only a title, and a fully described book. Every object must start with a valid title and a non-negative copy count. A private validation function is used by the constructors so that invalid copy counts are rejected in one place.

2. Implementation

Source: Task01/constructors.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class Book {
    string title;
    string author;
```

```
    int copies;

    static int validCopies(int n) {
        if (n < 0) throw invalid_argument("copies cannot be negative");
        return n;
    }
public:
    Book() : Book("Untitled", "Unknown", 0) {}
    explicit Book(const string& t) : Book(t, "Unknown", 1) {}
    Book(const string& t, const string& a, int n)
        : title(t), author(a), copies(validCopies(n)) {}

    void display() const {
        cout << title << " | " << author << " | copies: " << copies << '\\n';
    }
};

int main() {
    Book empty;
    Book donated("C++ Basics");
    Book full("Object Oriented Programming", "B. Stroustrup", 5);
    empty.display();
    donated.display();
    full.display();

    try { Book invalid("Bad record", "Unknown", -2); }
    catch (const invalid_argument& e) { cout << "Rejected: " << e.what() << '\\n'; }
}
```

3. Output

```
Untitled | Unknown | copies: 0
C++ Basics | Unknown | copies: 1
Object Oriented Programming | B. Stroustrup | copies: 5
Rejected: copies cannot be negative
```

4. Design Explanation

- The compiler supplies a default constructor only when the class declares no constructors. Once any constructor is declared, the compiler does not automatically add a no-argument constructor.
- A member initializer list constructs members directly. Assignment inside the constructor body first default-constructs members and then assigns new values; direct initialization is usually clearer and may be more efficient.
- Members are initialized in the order in which they are declared in the class, not the order written in the initializer list.
- Constructor delegation uses Book(...) to call the full constructor, avoiding duplicated validation and initialization logic.
- The title-only constructor is explicit. Without explicit, Book b = string("Novel"); could use an implicit conversion. With explicit, that copy-initialization is rejected; Book b("Novel"); remains valid.

Object	Constructor used	State after construction
empty	Book() delegates	Untitled, Unknown, 0
donated	Book(title) delegates	C++ Basics, Unknown, 1
full	Book(title, author, copies)	Object Oriented Programming, B. Stroustrup, 5

5. Required Experiments

If the default constructor is removed, a declaration such as Book shelf[3]; fails because the compiler must default-construct each array element. Typical diagnostic: “no matching function for call to Book::Book()”.

For the -Wreorder experiment, declare members as int copies; string title; but write an initializer list as Book(...) : title(t), copies(n). The compiler still initializes copies before title and -Wreorder warns that the list order differs from declaration order. Keep declaration and initializer order consistent in the final program.

Negative copies are tested in the program. The invalid_argument exception prevents an object from being created with a negative number of copies.

6. Alternative Concept and Complexity

Single constructor with default arguments

```
Book(string t = "Untitled", string a = "Unknown", int n = 0)
    : title(t), author(a), copies(validCopies(n)) {}
```

Both designs create one Book in O(1) time and O(1) auxiliary space (excluding dynamically managed string storage). A single constructor with defaults is shorter, but overloaded constructors communicate the three creation situations more clearly. Constructor delegation plus one validation function checks the copy rule in one place.

Conclusion: Constructors ensure that every Book begins in a valid state. Initializer lists and delegation reduce repeated code, while validation prevents invalid records.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

1. Problem Analysis

Prescription owns a dynamically allocated array of medicine names. The compiler-generated copy constructor copies the pointer value only. Therefore two objects point to the same array, causing changes to be shared and potentially causing a double delete. A deep copy allocates a separate array for each object.

2. Implementation – Rule of Three

Source: Task02/prescription.cpp

```
#include <iostream>
#include <string>
```

```
using namespace std;

class Prescription {
    int n;
    string* medicines;
public:
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new string[count] : nullptr) {}

    Prescription(const Prescription& other)
        : n(other.n), medicines(n > 0 ? new string[n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
        cout << "Copy constructor called\\n";
    }

    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;
        string* fresh = other.n > 0 ? new string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
    }

    ~Prescription() { delete[] medicines; }

    void set(int i, const string& m) { medicines[i] = m; }
    string get(int i) const { return medicines[i]; }
};

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol"); p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\\n';
    r = r; // self-assignment test
}
```

3. Output

```
Copy constructor called
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

The copy-constructor message appears once because q is copy-constructed from p. Assignment to r uses the copy-assignment operator, not the copy constructor. Self-assignment is safe because the operator checks this == &other.

4. Shallow and Deep Copy Memory Diagrams

```
SHALLOW COPY (unsafe)
p.medicines ----+
                +----> [ Paracetamol | ORS ]
q.medicines ----+

DEEP COPY (independent)
p.medicines ---------> [ Paracetamol | ORS ]
q.medicines ---------> [ Paracetamol | ORS ]  (separate allocation)
```

- Shallow copy duplicates the pointer, not the allocated array. Both destructors attempt delete[] on the same address, producing a double-free error.
- Deep copy allocates a new array and copies each medicine string. Changes to one prescription do not affect another.
- Rule of Three: when a class manually manages a resource and needs a destructor, it usually needs a copy constructor and copy-assignment operator too.
- The copy constructor is called when initializing an object from another object, when passing an object by value, and when returning an object by value if copy elision does not remove the copy.

5. Experimental Investigation

Compile the deep-copy version with: g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer prescription.cpp -o prescription. Run ./prescription. A correct run should finish without AddressSanitizer errors. Compile the shallow version separately with the same flags; when both objects delete the shared pointer, AddressSanitizer should report an attempting double-free error.

Add a copy counter to the copy constructor to count copies. Pass Prescription by value to force a copy (unless optimization rules apply to a returned temporary); pass by const Prescription& to avoid copying. Measure copying for 10, 1,000 and 100,000 medicines using std::chrono. The actual milliseconds depend on your computer and must be copied from your own run.

6. Alternative – std::vector and Complexity

```
class Prescription {
    std::vector<std::string> medicines;
public:
    explicit Prescription(int n) : medicines(n) {}
};
```

std::vector manages its memory and copies its elements independently. Its compiler-generated copy constructor performs a deep logical copy, so the class follows the Rule of Zero. Copying n medicine names takes O(n) time and O(n) additional space. A raw shallow pointer copy is O(1), but it is incorrect ownership and unsafe.

Conclusion: Use the Rule of Three for manually owned dynamic memory. Prefer std::vector when possible because it manages allocation, copying and destruction safely.

Q3 – Dynamic Constructors and Destructors (7 Marks)

1. Design and Implementation

Source: Task03/auditorium.cpp

```
#include <iostream>
using namespace std;

class Auditorium {
    int rows, seats, booked;
    bool** map;
public:
    Auditorium(int r, int s) : rows(r), seats(s), booked(0), map(nullptr) {
        if (rows <= 0 || seats <= 0) { rows = seats = 0; }
        else {
            map = new bool*[rows];
            int made = 0;
            try {
                for (; made < rows; ++made) {
                    map[made] = new bool[seats]{};
                }
            } catch (...) {
                while (made > 0) delete[] map[--made];
                delete[] map; map = nullptr; rows = seats = 0;
                throw;
            }
        }
        cout << "Event created: " << rows << " x " << seats << '\\n';
    }

    bool book(int r, int s) {
        if (!map || r < 0 || r >= rows || s < 0 || s >= seats || map[r][s])
            return false;
        map[r][s] = true; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (!map || r < 0 || r >= rows || s < 0 || s >= seats || !map[r][s])
            return false;
        map[r][s] = false; --booked; return true;
    }
    ~Auditorium() {
        if (map) {
            for (int r = 0; r < rows; ++r) delete[] map[r];
            delete[] map;
        }
        cout << "Event closed. Seats booked: " << booked << '\\n';
    }
};

int main() {
    Auditorium local(2, 3);
    local.book(0, 1); local.book(1, 2); local.cancel(0, 1);
    {
        Auditorium blockEvent(1, 2);
        blockEvent.book(0, 0);
```

```
    }
    Auditorium* heapEvent = new Auditorium(2, 2);
    heapEvent->book(1, 1);
    delete heapEvent;
    Auditorium zero(0, 5);
}
```

2. Representative Output

```
Event created: 2 x 3
Event created: 1 x 2
Event closed. Seats booked: 1
Event created: 2 x 2
Event closed. Seats booked: 1
Event created: 0 x 0
Event closed. Seats booked: 0
Event closed. Seats booked: 1
```

Destruction order is reverse construction order. The blockEvent is destroyed when its block ends. The heapEvent is destroyed only when delete heapEvent is executed. The local and zero objects are destroyed at the end of main, in reverse order: zero, then local.

3. Design Analysis and Memory Layout

```
map
 |
 +--> row pointer 0 --> [ seat ][ seat ][ seat ]
 +--> row pointer 1 --> [ seat ][ seat ][ seat ]

Allocations: 1 allocation for row pointers + one allocation per row
Total: rows + 1 allocations
```

- If delete is forgotten for an object created with new, its destructor is not called and its owned memory leaks.
- Use delete[] for arrays allocated with new[]. Using delete instead of delete[] is undefined behaviour.
- A destructor has no return type or parameters and cannot be overloaded because object destruction has one fixed signature.
- RAII ties resource release to object lifetime. When the object leaves scope or is deleted, its destructor releases its memory automatically.

4. Experimental Investigation

Run the correct program with -fsanitize=address. It should report no memory error. For the leak experiment, intentionally omit delete heapEvent; LeakSanitizer should report leaked allocations when enabled. Restore delete before submission.

Memory for a bool seat map is approximately rows × seats bytes for the seat values, plus rows × sizeof(bool*) bytes for row pointers and allocator overhead. Thus 10×20 is about 200 bytes of seat values; 100×200 is about 20,000 bytes; 1,000×2,000 is about 2,000,000 bytes, excluding overhead.

5. Alternative Design and Complexity

```
// One contiguous allocation
std::vector<bool> seats(rows * perRow);
// Access seat (r, s) as seats[r * perRow + s].
```

Access by index is O(1) in all designs. A row-pointer design uses rows+1 allocations and stores an extra row-pointer array. One contiguous block needs one allocation and has better locality. std::vector<std::vector<char>> is convenient but normally uses one vector allocation plus one allocation per row.

Conclusion: Dynamic construction allows each event to have its own seat dimensions. A destructor and RAII prevent memory leaks when objects leave their lifetime.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

1. Design and Implementation

Source: Task04/money.cpp

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
using namespace std;

class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    Money operator+(const Money& other) const {
        return Money(0, static_cast<int>(paise + other.paise));
    }
    Money operator-(const Money& other) const {
        return Money(0, static_cast<int>(paise - other.paise));
    }
    Money& operator+=(const Money& other) {
        paise += other.paise; return *this;
    }
    bool operator==(const Money& other) const { return paise == other.paise; }
    bool operator<(const Money& other) const { return paise < other.paise; }
    friend ostream& operator<<(ostream& os, const Money& m) {
        long long whole = m.paise / 100;
        long long cents = m.paise % 100;
        if (cents < 0) cents = -cents;
        return os << "Rs " << whole << '.' << setw(2) << setfill('0') << cents
                  << setfill(' ');
    }
    friend istream& operator>>(istream& is, Money& m) {
        long long r; int p;
        if (is >> r >> p) m.paise = r * 100 + p;
        return is;
    }
};
```

```
int main() {
    Money a(120,75), b(79,25);
    Money c = a + b;
    cout << c << '\\n' << boolalpha << (c == Money(200)) << '\\n';
    Money x(0,75), y(0,50);
    cout << (x + y) << '\\n';
    cout << (Money(1,0) - Money(2,0)) << '\\n';
}
```

2. Output

```
Rs 200.00
true
Rs 1.25
Rs -1.00
```

The representation stores the total number of paise in a long long, so addition and comparison use exact integer arithmetic. This implementation permits negative balances for subtraction; a billing application may instead reject a negative result according to its business rule.

3. Operator Design Explanation

- operator+ returns a new Money object because a + b should not modify either operand.
- operator+= modifies the left operand and returns Money& (*this) so chained expressions such as a += b += c work without an unnecessary copy.
- operator<< is a non-member because the left operand is an ostream, not a Money object. It is a friend so it can read the private paise member.
- operator>> is also a friend non-member for the same reason: the left operand is istream.
- Member operators are suitable for +, -, +=, == and < in this design. A friend/non-member form is useful when implicit conversions should work symmetrically.
- The expression a + b is interpreted as a.operator+(b) for this member implementation.

4. Engineering Decision and Complexity

Multiplication of one Money amount by another Money amount is generally not meaningful: the result would have squared currency units. Multiplication by a numeric quantity (for example, price × number of items) is meaningful and may be added as a separate overload.

Every arithmetic and comparison operation is O(1) time and O(1) extra space. A double-based version can accumulate rounding errors because many decimal fractions cannot be represented exactly in binary floating point. Integer paise gives exact two-decimal currency arithmetic. For the required million-addition comparison, use a loop with std::chrono for both implementations and record the times printed on your own computer.

Conclusion: Operator overloading makes billing expressions natural while integer paise avoids floating-point currency errors.

Q5 – Increment, Comparison and Subscript Operators (6 Marks)

1. Implementation

Source: Task05/time_playlist.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

class Time {
    int seconds;
public:
    explicit Time(int s=0) : seconds(s) {
        if (s < 0) throw invalid_argument("negative time");
    }
    Time& operator++() { ++seconds; return *this; }       // prefix
    Time operator++(int) { Time old=*this; ++seconds; return old; } // postfix
    bool operator==(const Time& t) const { return seconds==t.seconds; }
    bool operator<(const Time& t) const { return seconds<t.seconds; }
    friend ostream& operator<<(ostream& os, const Time& t) {
        int h=t.seconds/3600, m=(t.seconds/60)%60, s=t.seconds%60;
        return os << h << ':' << (m<10?"0":"") << m << ':' << (s<10?"0":"") << s;
    }
};

class Playlist {
    vector<string> tracks;
public:
    Playlist(initializer_list<string> x) : tracks(x) {}
    string& operator[](int i) {
        if (i<0 || i>=static_cast<int>(tracks.size())) throw out_of_range("track index");
        return tracks[i];
    }
    const string& operator[](int i) const {
        if (i<0 || i>=static_cast<int>(tracks.size())) throw out_of_range("track index");
        return tracks[i];
    }
    int size() const { return static_cast<int>(tracks.size()); }
};

int main() {
    Time t(3598);
    Time a=t++;
    Time b=++t;
    cout << a << " | " << b << " | " << t << '\\n';
    Playlist p{"Song A","Song B","Song C"};
    cout << p[0] << " / " << p[2] << '\\n';
    try { cout << p[p.size()] << '\\n'; }
    catch (const out_of_range& e) { cout << "Error: " << e.what() << '\\n'; }
}
```

2. Output

```
0:59:58 | 1:00:00 | 1:00:00
Song A / Song C
Error: track index
```

Prefix ++ increments first and returns the updated object by reference. Postfix ++ has a dummy int parameter so the compiler can distinguish it; it returns the old value by copy. The const subscript overload allows read-only playlists to be indexed, while the non-const overload permits changing a track.

3. Predict the Output – Counter Program

Prediction and actual output

```
5 7 7
```

Initially c is 5. a = c++ copies the old value 5 into a and then c becomes 6. b = ++c increments c to 7 and returns the updated value, so b is 7 and c is 7. Prediction and actual output should match.

4. Experimental Investigation and Complexity

Test playlist[-1] and playlist[size]; both should throw std::out_of_range. The Time class stores total seconds, so 59:59 followed by ++ becomes 1:00:00 automatically. For 10,000,000 increments, compile at -O0 and -O2 and measure prefix and postfix loops with std::chrono. Record your actual measurements; the compiler may optimize simple loops, so use a checksum/observable result.

Prefix increment is preferred for class types because it avoids creating and returning the old temporary object. Playlist access is O(1). A bounds check adds one comparison and a branch, still O(1). An at(i) method can throw std::out_of_range; unchecked access is faster only marginally but can cause undefined behaviour.

Conclusion: Prefix increment avoids an unnecessary copy. Checked subscript access makes playlist operations safer by detecting invalid indices.

Q6 – The Rules of Operator Overloading (7 Marks)

1. Implementation

Source: Task06/text_class.cpp

```
#include <cstring>
#include <iostream>
using namespace std;

class Text {
    char* data;
public:
    Text(const char* s="") {
        data = new char[strlen(s)+1];
        strcpy(data,s);
    }
    Text(const Text& other) {
```

```
        data = new char[strlen(other.data)+1];
        strcpy(data,other.data);
        cout << "[copy] ";
    }
    Text& operator=(const Text& other) {
        if (this == &other) return *this;
        char* fresh = new char[strlen(other.data)+1];
        strcpy(fresh,other.data);
        delete[] data;
        data=fresh;
        return *this;
    }
    ~Text() { delete[] data; }

    Text operator+(const Text& other) const {
        size_t n=strlen(data)+strlen(other.data);
        char* temp=new char[n+1];
        strcpy(temp,data); strcat(temp,other.data);
        Text result(temp);
        delete[] temp;
        return result;
    }
    bool operator==(const Text& other) const { return strcmp(data,other.data)==0; }
    bool operator!=(const Text& other) const { return !(*this==other); }
    friend ostream& operator<<(ostream& os,const Text& t) { return os << t.data; }
};

int main() {
    Text a("Hello "), b("C++"), c("!");
    Text message = a + b + c;
    cout << '\\n' << message << '\\n';
    Text empty;
    cout << "empty=[" << empty << "]\\n";
    message = message; // self-assignment
    cout << boolalpha << (message != a) << '\\n';
}
```

2. Output

```
[copy]
Hello C++!
empty=[]
true
```

The exact number of copy messages can depend on copy elision and compiler behavior. The message shows when the explicit copy constructor is invoked; compile with the same compiler and flags used for your report.

3. Operator Overloading Rules

- Operators that cannot be overloaded include ::, ., .*, ?: and sizeof.
- At least one operand must be a user-defined type. An operator cannot be redefined for two built-in operands only.

- Overloading cannot change an operator's precedence, associativity or number of operands (arity).
- operator= must be a non-static member function because assignment is defined as an operation on the left-hand object.
- A friend is useful when a non-member operator needs access to private data, such as stream insertion where the left operand is ostream.

4. Experimental Investigation

Compile with -fsanitize=address and run. Correct execution should produce no memory errors. Test self-assignment (message = message), empty strings, and long messages. Compare 100,000 concatenations against std::string using std::chrono and record actual times. Avoid repeatedly building a long result with + in a loop if the purpose is to compare efficient append operations; compare equivalent workloads.

5. Alternative Concept and Complexity

```
// Preferred library alternative
std::string a = "Hello ", b = "C++", c = "!";
std::string message = a + b + c;

// Alternative operator+ implemented through +=
Text operator+(Text left, const Text& right) {
    left += right;
    return left;
}
```

For a + b + c, the first concatenation copies n1+n2 characters and the second copies n1+n2+n3 characters, so the total character work is O(n1+n2+n3) for this two-step expression, with temporary storage for intermediate results. std::string manages ownership and provides optimized operations. A custom Text class is useful when learning ownership or implementing specialized storage, but production code should normally use std::string.

Conclusion: Writing Text demonstrates dynamic memory ownership, deep copying, self-assignment protection and operator design. Standard library string is safer and more maintainable for normal application development.

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the Problem

An AC circuit impedance is represented by Z = R + jX, where R is the real part and X is the imaginary part. The program must construct complex values, add/subtract/multiply/divide them, compare them, print rectangular form and calculate magnitude. It must reject division by zero.

Requirement	Measurable statement
Representation	Store one real and one imaginary double.
Construction	Support zero, real-only and real-plus-imaginary values.

Arithmetic	Implement +, -, *, / using complex-number formulas.
Display	Print rectangular form and magnitude.
Safety	Detect zero denominator before division.

Stage 2 – Design and Formulas

For z1 = a + jb and z2 = c + jd:

```
Addition:       (a+c) + j(b+d)
Subtraction:    (a-c) + j(b-d)
Multiplication: (ac-bd) + j(ad+bc)
Division:       ((ac+bd)/(c*c+d*d)) + j((bc-ad)/(c*c+d*d))
Magnitude:      sqrt(a*a + b*b)
```

Arithmetic and comparison operators are members in this design. Stream insertion is a friend non-member because ostream is the left operand. The constructors are default, real-only and real-plus-imaginary.

Stage 3 – Implementation and Verification

Source: Task07/complex_impedance.cpp

```
#include <cmath>
#include <iostream>
#include <stdexcept>
using namespace std;

class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}
    Complex(double real) : re(real), im(0) {}
    Complex(double real, double imag) : re(real), im(imag) {}

    Complex operator+(const Complex& z) const {
        return {re+z.re, im+z.im};
    }
    Complex operator-(const Complex& z) const {
        return {re-z.re, im-z.im};
    }
    Complex operator*(const Complex& z) const {
        return {re*z.re-im*z.im, re*z.im+im*z.re};
    }
    Complex operator/(const Complex& z) const {
        double d=z.re*z.re+z.im*z.im;
        if (d == 0.0) throw domain_error("division by zero impedance");
        return {(re*z.re+im*z.im)/d, (im*z.re-re*z.im)/d};
    }
    bool operator==(const Complex& z) const {
        const double eps=1e-9;
        return abs(re-z.re)<eps && abs(im-z.im)<eps;
    }
    double magnitude() const { return sqrt(re*re+im*im); }
    friend ostream& operator<<(ostream& os,const Complex& z) {
```

```
        return os << z.re << (z.im<0 ? " - j" : " + j") << abs(z.im);
    }
};

int main() {
    const double R=10.0, XL=5.0, XC=10.0;
    Complex resistor(R,0), inductor(0,XL), capacitor(0,-XC);
    Complex series=resistor+inductor+capacitor;
    Complex parallel=(resistor*inductor)/(resistor+inductor);
    cout << "Series impedance: " << series << '\\n';
    cout << "Series magnitude: " << series.magnitude() << '\\n';
    cout << "Parallel R-L: " << parallel << '\\n';
    try {
        Complex invalid = resistor / Complex(0,0);
        cout << invalid << '\\n';
    } catch (const domain_error& e) {
        cout << "Error: " << e.what() << '\\n';
    }
}
```

Expected Output

```
Series impedance: 10 - j5
Series magnitude: 11.1803
Parallel R-L: 2 + j4
Error: division by zero impedance
```

Hand Verification

Series: Z = 10 + j5 − j10 = 10 − j5. Magnitude = √(10² + (−5)²) = √125 = 11.1803. For the parallel R-L example, (10 × j5)/(10 + j5) = j50/(10+j5). Multiplying numerator and denominator by (10−j5) gives (250+j500)/125 = 2+j4. Division by 0+j0 is rejected.

Stage 4 – Defend the Design

- The default constructor represents zero impedance; the real-only constructor represents a resistor; the two-part constructor represents a general impedance.
- Each arithmetic operator corresponds to the standard complex-number formula. Stream insertion makes results easy to display.
- std::complex<double> already provides tested arithmetic and magnitude operations and is preferred in production software. A custom class is useful for learning, custom validation and specialized display.
- Floating-point equality is approximate because decimal values may have representation errors. This implementation uses a small tolerance; the tolerance must be selected for the application's scale.
- One limitation is that the example uses exact zero detection for the division denominator and does not model frequency-dependent component tolerances or measurement uncertainty.

Alternative Concept and Complexity Analysis

Using std::complex<double> would reduce implementation work: include <complex>, construct std::complex<double> values and use +, -, *, / and std::abs. Time per arithmetic operation is O(1) for

both designs. Both store two doubles, so storage is O(1). The standard library implementation is well tested; explicit zero-denominator handling is still required by the application.

Conclusion: The Complex class converts circuit impedance formulas into readable C++ operations. It supports series and parallel calculations and protects the program from division by zero.

Overall Submission Checklist

- Create the GitHub repository named DSA01-192471055-Assignment3 and add the working URL to the cover page.
- Save one source file (or clearly named files) for every task in its own folder.
- Compile every program using g++ -std=c++17 -Wall -Wextra and resolve all warnings that belong to the final version.
- Run sanitizer experiments for Tasks 2 and 3 and Task 6; include screenshots or terminal logs from your own execution.
- Run the prediction programs before finalizing the predicted-output section and retain the actual terminal output.
- Run all timing experiments on your own computer at -O0 and -O2 and replace the instruction to record timings with your measured values.
- Commit each task progressively and include README.md, input files if any, and output screenshots.

Important academic note: The deterministic output blocks in this document are reference outputs based on the supplied code. Timing measurements, compiler diagnostics and sanitizer messages vary by compiler, operating system and run. Execute them yourself and attach your genuine terminal evidence before submission.